# Tutorial 05 — Instructor notes

ENVX2001 – Applied Statistical Methods

Semester 1

# Contents



Companion document for tutors running Tutorial 05. This tutorial is a walkthrough — you lead every exercise from start to finish. The only section students work on independently is the exam questions at the end.

Student document: Tutorial 05

# Before you start

Get these sorted before students arrive:

- Have RStudio open with a fresh session. No special data files are needed — all data is simulated in the tutorial code.
- Make sure `ggplot2` is installed. No other packages are required.
- Count the number of students present. You will need this for Exercise 1.
- Skim the three exercises: Exercise 1 is a class activity on randomisation (~15 min), Exercise 2 walks through a CRD analysis (~10 min), Exercise 3 walks through an RCBD analysis with variance partitioning (~15 min). Budget about 40 minutes total.

# Exercise 1: Testing randomisation

This is a live class activity. You run the non-random sample first (whole class), then randomly select a subset of students and repeat the question.

## Part A — Non-random sample

### What to do

Ask every student: "Do you know the student sitting next to you? Yes or No." Have students raise their hand for Yes and keep it down for No. Count the Yes and No responses.

Update the code with the actual class numbers:

```
CODE
total_nonrandom ← 60 # replace with actual class size
count_yes_nonrandom ← 38 # replace with actual Yes count
count_no_nonrandom  ← 22 # replace with actual No count

class_size ← total_nonrandom
```

### What to say

- This is our non-random (convenience) sample. We asked everyone, but the question itself introduces bias — students who sit next to friends will answer differently from those who sat in a random spot.
- Point out that the sample is "non-random" in the sense that seating arrangements are not random. Friends tend to cluster.

## Part B — Percentages

### What to do

Walk through the percentage calculation. Run the code live.

```
CODE
prop_yes_nonrandom ← (count_yes_nonrandom / total_nonrandom) * 100
prop_no_nonrandom  ← (count_no_nonrandom  / total_nonrandom) * 100

cat("Non-random sample % Yes:", round(prop_yes_nonrandom, 1), "%\n")
cat("Non-random sample % No :", round(prop_no_nonrandom, 1),  "%\n\n")
```

### What to say

- Typically the Yes percentage is high (60-70%), because friends sit together. This is the bias we want to demonstrate.

## Part C — Random sample

### What to do

Run the random selection code to pick a subset of students. Call out the selected student numbers. Ask only those students the question again, and record the counts.

```
CODE
set.seed(123)
student_ids ← seq_len(class_size)
rand_num ← runif(class_size)
class_df ← data.frame(id = student_ids, rand_num = rand_num)

random_sample_size ← 30

set.seed(202)
sampled_ids ← sample(class_df$id, size = random_sample_size, replace = FALSE)
sampled_ids
```

Update the random sample counts and run the comparison:

```
CODE
count_yes_random ← 14 # replace with actual count
count_no_random  ← 16 # replace with actual count

prop_yes_random ← (count_yes_random / random_sample_size) * 100
prop_no_random  ← (count_no_random  / random_sample_size) * 100

cat("Random sample % Yes:", round(prop_yes_random, 1), "%\n")
cat("Random sample % No :", round(prop_no_random, 1),  "%\n\n")
```

Then compute the bias and show the bar plot:

```
CODE
```

```
bias_yes ← prop_yes_nonrandom - prop_yes_random
bias_no  ← prop_no_nonrandom  - prop_no_random

cat("Estimated bias in % Yes (non-random - random):", round(bias_yes, 1), "%\n")
cat("Estimated bias in % No  (non-random - random):", round(bias_no, 1),  "%\n\n")
```

CODE

```
percent_df ← data.frame(
  sample_type = rep(c("Non-random", "Random"), each = 2),
  response    = rep(c("Yes", "No"), times = 2),
  percent     = c(prop_yes_nonrandom, prop_no_nonrandom, prop_yes_random, prop_no_random)
)

library(ggplot2)

ggplot(percent_df, aes(x = response, y = percent, fill = sample_type)) +
  geom_col(position = "dodge") +
  labs(x = "Response", y = "Percent", fill = "Sample type")
```

### What to say

- The random sample typically has a lower Yes percentage than the non-random sample. The difference is our estimate of bias.
- If the percentages happen to be similar, that is fine — it does not mean randomisation failed. It means seating was not very clustered in this particular class. The principle still holds.
- The key takeaway: non-random sampling (convenience sampling) can systematically over- or under-represent certain groups. Random sampling gives every unit an equal chance of selection, which reduces systematic bias.
- Connect to experimental design: randomisation in experiments works the same way. If we do not randomly assign treatments, systematic differences between groups can confound our results.

### Common issues

> ⚠ Warning
>
> **Students do not know their number.** If you did not assign numbers at the start, you can number students by row and seat position (e.g. "row 3, seat 5 is student 17"). Alternatively, skip the physical selection and just use the pre-filled example data.

> ⚠ Warning
>
> **The bias is small or reversed.** This can happen with smaller classes or when seating is genuinely mixed. Use it as a teaching moment — randomisation does not guarantee a perfect result every time, but on average it produces less biased estimates.

# Exercise 2: CRD

This exercise walks through a simulated CRD experiment. You lead the entire exercise — students follow along and run each code block.

### What to do

Walk through the CRD concept using the diagram (Slide1.PNG), then work through the simulated experiment step by step:

1. Show the statistical model and explain each term.
2. Run the treatment assignment code and data simulation code (both are code-folded — expand them).
3. Ask students: "Is there a significant difference?" before showing the ANOVA output.
4. Run the ANOVA and interpret the result together.

CODE

```
set.seed(123)
treatments ← rep(c("A", "B", "C"), each = 4)
pots ← data.frame(pot_id = 1:12, treatment = sample(treatments))

set.seed(456)
plant_heights ← data.frame(
  pot_id = rep(1:12, each = 5),
  plant_id = 1:60,
  height = rnorm(60, mean = rep(c(10, 15, 20), each = 20), sd = 2)
)
plant_heights ← merge(plant_heights, pots, by = "pot_id")

anova_result ← aov(height ~ treatment, data = plant_heights)
summary(anova_result)
```

### What to say

- **Statistical model:** $Y_{ij} = \mu + \alpha_i + \epsilon_{ij}$. Walk through what each symbol means in plain language. The "word version" (plant height = overall mean + fertilizer treatment + random error) is there to help students who find the notation intimidating.
- **The simulation:** We are generating fake data where treatment A has a true mean of 10, B has 15, and C has 20. The ANOVA should easily detect these differences. This is deliberate — the point is to practise reading the output, not to deal with ambiguous results.
- **ANOVA output:** The treatment F-statistic will be large and the p-value very small. Point out: large F means the between-group differences are much bigger than the within-group noise. That is all F measures.
- **Experimental unit:** Each pot is the experimental unit (treatment is applied to the pot). Each plant is a measurement unit. With 5 plants per pot, there are 12 experimental units but 60 observations. In a real analysis we would need to account for this — but for now the simulated data ignores the nesting to keep things simple.

### Common issues

> ⚠ Warning
>
> **"Why are there 5 plants per pot but we are not averaging them?"** Good question. In a real experiment with sub-sampling, you would either average within pots or use a nested/mixed model. This simulation treats each plant as independent to keep the ANOVA straightforward. Acknowledge the shortcut and mention that more realistic designs are covered in later weeks.

# Exercise 3: RCBD

This exercise extends the CRD by adding blocks. You lead the entire exercise.

### What to do

Walk through the RCBD concept using the diagram (Slide2.PNG), then work through the simulated experiment:

1. Show the statistical model — highlight the extra $\beta_j$ (block) term compared to the CRD model.
2. Run the block assignment, data simulation, and ANOVA code.
3. Walk through the percentage of variation explained calculation.
4. Ask students to try the hand calculation before showing the R code.

CODE

```r
set.seed(123)
blocks ← rep(c("North", "South", "East", "West"), each = 3)
treatments ← rep(c("A", "B", "C"), times = 4)
pots ← data.frame(pot_id = 1:12, block = blocks, treatment = sample(treatments))

set.seed(456)
plant_heights ← data.frame(
  pot_id = rep(1:12, each = 5),
  plant_id = 1:60,
  height = rnorm(60, mean = rep(c(10, 15, 20), each = 20) + rep(c(2, -2, 1, -1), each = 15), sd =
2)
)
plant_heights ← merge(plant_heights, pots, by = "pot_id")

anova_result ← aov(height ~ block + treatment, data = plant_heights)
summary(anova_result)
```

For the variance partitioning:

CODE

```r
```

```r
anova_table ← summary(anova_result)[[1]]
ss_total ← sum(anova_table$`Sum Sq`)
ss_treatment ← anova_table$`Sum Sq`[2]
ss_block ← anova_table$`Sum Sq`[1]
percent_treatment ← (ss_treatment / ss_total) * 100
percent_block ← (ss_block / ss_total) * 100
cat("Percentage of variation explained by treatments:", round(percent_treatment, 1), "%\n")
cat("Percentage of variation explained by blocks:", round(percent_block, 1), "%\n")
```

### What to say

- **CRD vs RCBD:** The only difference in R is adding `block` to the formula before `treatment`. The block term absorbs variation that would otherwise end up in the residual, making it easier to detect treatment effects.
- **Model formula order matters:** `aov(height ~ block + treatment)` tests block first, then treatment. In a balanced design (equal numbers in each cell) the order does not change the treatment F-test. Mention this briefly but do not go deep — it becomes important in unbalanced designs later.
- **Percentage of variation explained:** This is SS for a source divided by total SS, times 100. Walk through one calculation by hand from the ANOVA table before showing the R code. Emphasise that students need to be able to do this with a calculator in assessments.
- **Is blocking useful?** Look at the block p-value. If it is significant, blocking explained real variation that would have inflated the residual in a CRD. If it is not significant, blocking was unnecessary (but did not hurt).
- **Connect to the lab:** In the lab this week, students will analyse real CRD and RCBD data (dingo/fox camera traps and fire/mammal abundance). The tutorial has given them practice with the R code and model interpretation on simulated data so the lab is not their first time seeing it.

### Common issues

> ⚠ Warning
>
> **Students confuse the block and treatment rows in the ANOVA table.** The order in the table matches the formula: block appears first, treatment second. Point to the row names explicitly when reading the output.

> ⚠ Warning
>
> **"Why do we not test interactions between block and treatment?"** In an RCBD we assume no interaction between blocks and treatments. This is a simplifying assumption. If the interaction were real, we would need more replicates per cell to estimate it. This comes up later in the course with factorial designs.

⚠ Warning

**Hand calculation errors with SS.** Students sometimes use the wrong rows or forget to sum all SS for the total. Walk through it slowly: Total SS = Block SS + Treatment SS + Residual SS. Then percentage = (source SS / Total SS) × 100.

# Wrapping up

## What to say

- Three ideas from today: (1) randomisation reduces bias in both sampling and experiments; (2) a CRD randomly assigns treatments to homogeneous units; (3) an RCBD adds blocks to account for known sources of variation, which reduces residual noise and makes treatment effects easier to detect.
- The exam questions at the end of the tutorial are for self-study. Do not walk through them in class, but encourage students to attempt them before the lab.
- In the lab, students will work with real data (camera trap surveys and fire ecology) using the same `aov()` and variance partitioning workflow they practised here.